Guía 1 de 3 — Leer y guardar archivos: CSV, Excel, JSON y Parquet

DPL1046 · Clase 4 · Para repasar en casa, a tu ritmo.

Los datos llegan en archivos de formatos distintos. Un buen ingeniero de datos sabe **leer cada formato** y elegir **cuál usar para guardar**. Esta guía repasa los cuatro que vimos en clase, con ejemplos simples y las tablas de Olist.

> Antes de empezar, haz tu rutina de inicio (Guía 1 de la Clase 2): terminal, carpeta del repo, `code .`, activar `.venv` y `git pull`. Todos los ejemplos usan `import pandas as pd` y la carpeta `data/raw/`.

---

1. CSV: una lista escrita en el Bloc de notas

Imagina una lista de asistencia en el Bloc de notas:

```
nombre,edad
Ana,30
Luis,25
```

Cada línea es una fila y los datos se separan con una **coma**. Eso es un **CSV** (*comma-separated values*): texto plano, sin colores ni fórmulas. La primera línea trae los **nombres de las columnas**.

```
import pandas as pd

RUTA = "data/raw/"
pagos = pd.read_csv(RUTA + "olist_order_payments_dataset.csv")

print(pagos.shape)            # (103886, 5): filas y columnas
print(list(pagos.columns))    # los nombres, como lista
print(pagos.head(3))          # las 3 primeras filas
```

- `RUTA + "olist..."` **pega** dos textos y forma la dirección del archivo.
- `pd.read_csv(...)` abre el archivo y lo convierte en una **tabla** de pandas.
- `shape` = (filas, columnas). `head(3)` = las 3 primeras filas.

Leer solo una parte (archivos grandes)

```
liviana = pd.read_csv(
    RUTA + "olist_order_payments_dataset.csv",
    usecols=["order_id", "payment_value"],   # solo estas 2 columnas
    nrows=5,                                  # solo las 5 primeras filas
)
```

Es como pedir en la biblioteca "solo las primeras cinco páginas y dos columnas". Útil para **espiar** un archivo enorme sin cargarlo entero.

Si el CSV usa punto y coma

Los CSV exportados desde Excel en Chile suelen separar con `;`. Si al leer ves **una sola columna gigante**, es esto:

```
tabla = pd.read_csv("archivo.csv", sep=";")
```

---

2. Excel: un cuaderno con pestañas

Un archivo Excel (`.xlsx`) es como un **cuaderno con pestañas** ("Ventas", "Clientes"). Los reportes de gerencia llegan así. Hay que hacer dos cosas: **ver qué pestañas hay** y **leer una pestaña por su nombre**. pandas usa por detrás la biblioteca `openpyxl`, que ya está instalada.

Crear un Excel de ejemplo

```
ventas = pd.DataFrame({
    "region": ["RM", "Biobío", "Valparaíso"],
    "ventas": [1200, 800, 950],
})
clientes = pd.DataFrame({
    "region": ["RM", "Biobío", "Valparaíso"],
    "clientes": [300, 150, 200],
})

with pd.ExcelWriter("data/reporte_demo.xlsx") as escritor:
    ventas.to_excel(escritor, sheet_name="Ventas", index=False)
    clientes.to_excel(escritor, sheet_name="Clientes", index=False)
```

- `pd.DataFrame({...})` crea una **tabla a partir de un diccionario**: cada clave es una columna y cada lista son sus valores (lo de la Clase 3).
- `with ... as escritor:` abre el archivo para escribir y lo **cierra solo** al terminar el bloque. Los dos puntos y la **sangría** marcan qué pertenece al `with`.
- `to_excel(..., sheet_name="Ventas")` escribe la tabla en una pestaña. `index=False` evita guardar la columna 0, 1, 2 que pandas agrega.

Leer un Excel

```
archivo = pd.ExcelFile("data/reporte_demo.xlsx")
print(archivo.sheet_names)        # ['Ventas', 'Clientes']

hoja = pd.read_excel("data/reporte_demo.xlsx", sheet_name="Ventas")
print(hoja)
```

`ExcelFile` abre el cuaderno y mira el índice; `read_excel(..., sheet_name="Ventas")` lee **una** pestaña.

---

3. JSON: fichas con listas adentro

JSON es el formato en que responden las **APIs**. Usa lo que ya conoces: **diccionarios** `{ }` y **listas** `[ ]`, uno dentro de otro.

```
respuesta = {
    "marketplace": "Olist",
    "vendedores": [
        {"vendedor": "Tienda A", "estado": "SP", "ventas": 50000},
        {"vendedor": "Tienda B", "estado": "RJ", "ventas": 32000},
    ],
}
```

Piensa en una **dirección postal**: edificio → piso → departamento → buzón. Para llegar a un dato se entra **de a un nivel**, con un corchete por cada nivel:

Expresión	Qué entrega
`respuesta["marketplace"]`	`"Olist"`
`respuesta["vendedores"]`	la lista con los 2 vendedores
`respuesta["vendedores"][0]`	el primer vendedor (un diccionario)
`respuesta["vendedores"][0]["vendedor"]`	`"Tienda A"`

> **Regla simple:** entre comillas es una **clave**; un número es una **posición** (se cuenta desde 0).

De JSON a tabla

```
tabla = pd.json_normalize(respuesta["vendedores"])
print(tabla)
```

`json_normalize` convierte una **lista de diccionarios** en una **tabla**: cada diccionario pasa a ser una fila y cada clave una columna. Ojo: se le pasa **la lista** (`respuesta["vendedores"]`), no todo el JSON.

---

4. Parquet: la maleta bien hecha

Un CSV es como un libro escrito a mano: lo abre cualquiera, pero ocupa espacio. **Parquet** guarda el mismo contenido **comprimido y ordenado por columna**: ocupa menos y se abre más rápido. Es un archivo **binario** (no se abre con el Bloc de notas). Es el formato preferido en la nube y el que usaremos para **guardar resultados**. Requiere la biblioteca `pyarrow`.

```
import os

pagos.to_parquet("data/pagos.parquet", index=False)      # guardar
leido = pd.read_parquet("data/pagos.parquet")            # leer
print("Filas leídas:", len(leido))                       # 103886: no se perdió
nada

tam_csv = os.path.getsize(RUTA + "olist_order_payments_dataset.csv") / 1_000_000
tam_pq = os.path.getsize("data/pagos.parquet") / 1_000_000
print("CSV (MB):", round(tam_csv, 1), "· Parquet (MB):", round(tam_pq, 1))
```

- `os.path.getsize` entrega el tamaño en **bytes**; dividir por un millón lo pasa a **MB**. (El `_` en `1_000_000` solo ayuda a leer.)
- Resultado aproximado: el CSV pesa **5,8 MB** y el Parquet **3,8 MB**.

¿Qué formato uso?

Formato	Se abre con	Ventaja	Úsalo para
CSV	Bloc de notas, Excel	Universal	Recibir datos y compartir
Excel	Excel	Varias pestañas	Reportes de gerencia
JSON	Editor de texto	Datos anidados	Respuestas de APIs
Parquet	Solo programas	Liviano y rápido	**Guardar tus resultados**

Errores frecuentes

Qué ves	Por qué pasa	Qué hacer
`FileNotFoundError`	La ruta está mal o no estás en la carpeta del repo	Revisa el nombre y que la terminal esté en la carpeta raíz del curso
Una sola columna enorme	El CSV usa `;`	Agrega `sep=";"`
`ModuleNotFoundError: openpyxl` / `pyarrow`	Falta instalar la biblioteca	Activa el `.venv` y ejecuta `pip install -r requirements.txt`
`KeyError: 'vendedores'`	La clave no existe (mayúsculas, tildes)	Imprime el diccionario y copia la clave exacta
`TypeError: list indices must be integers`	Usaste una clave donde iba una posición	Recuerda: número = posición, comillas = clave

Practica tú

1. Lee `olist_orders_dataset.csv` y muestra su `shape`. ¿Cuántos pedidos hay?
2. Lee solo las columnas `order_id` y `order_status`, y solo 10 filas.
3. Con el diccionario `respuesta` de arriba, ¿cómo pides el **estado del segundo** vendedor?
4. Guarda la tabla de pedidos en Parquet y compara el tamaño con el CSV.

▼ Ver respuestas

1. `pd.read_csv("data/raw/olist_orders_dataset.csv").shape` → `(99441, 8)`: 99.441 pedidos.
2. `pd.read_csv(RUTA + "olist_orders_dataset.csv", usecols=["order_id", "order_status"], nrows=10)`
3. `respuesta["vendedores"][1]["estado"]` → `"RJ"`.
4. `orders.to_parquet("data/orders.parquet", index=False)` y luego `os.path.getsize(...)` en cada archivo.

Para llevarte

- **CSV**: texto plano, el más común. `read_csv`.
- **Excel**: cuaderno con pestañas. `ExcelFile` + `read_excel(sheet_name=...)`.
- **JSON**: diccionarios y listas anidados. Un corchete por nivel; `json_normalize` para pasarlo a tabla.
- **Parquet**: liviano y rápido; para guardar resultados. `to_parquet` / `read_parquet`.